Universität Bremen

Minimale Neuberechnung Abhängigkeits-Graph basierter Regeln zur prozeduralen Welten-Generation

Bachelorarbeit zur Erlangung des akademischen Grades
Bachelor of Science (B.Sc.) in Informatik

Eingereicht von

Maarten Behn

behn@uni-bremen.de

Prüfer

Prof. Dr. Gabriel Zachmann
Prof. Dr. Nico Hochgeschwender

FB3

Uni Bremen

SoSe 2026

20. April 2026

1 Einleitung

Prozedurale Generierung ist ein etabliertes Verfahren zur automatischen Erstellung komplexer virtueller Welten, das insbesondere in der Spieleentwicklung, Simulation und Computergrafik Anwendung findet. Dabei werden zunächst grobe Strukturen erzeugt, die anschließend schrittweise durch immer feinere Details ergänzt werden. Wird der Generationsalgorithmus in irgendeiner Form angepasst, kann es vorkommen, dass eine bereits erzeugte Welt nicht mehr dem aktuellen Stand des Algorithmus entspricht.

Die Rechenzeit eines prozeduralen Generationsalgorithmus steigt mit der Menge der erzeugten Details. Insbesondere bei komplexen Algorithmen kann die vollständige Neugenerierung einer Welt sehr zeitaufwendig sein. Dies stellt ein Problem dar, da in der Praxis Generationsalgorithmen häufig iterativ entwickelt und angepasst werden. Lange Neugenerationszeiten können hier sehr stören.

Ziel dieser Arbeit ist es, zu untersuchen, inwieweit sich die Neugenerationszeit einer prozeduralen Welt verkürzen lässt, wenn nur diejenigen Teile der Welt neu berechnet werden, die durch Änderungen am Generationsalgorithmus ungültig geworden sind.

Dazu wird ein System vorgestellt, welches einen Generationsalgorithmus als Abhängigkeitsgraph darstellt. Dieses verwendet Zwischenergebnisse für diejenigen Teile des Graphen, die sich nicht geändert haben, wieder.

Inhaltsverzeichnis

1 Einleitung **1**
2 Stand der Technik **3**
2.1 Noise & Zufälligkeit 3
2.2 Example based Model Synthesis 3
2.3 Graph Grammatiken 5
2.4 KI basierte prozedurale Generation 6
2.5 Houdini & Blender Geometry Nodes 7
3 Theoretische Grundlagen **8**
3.1 Minimales Berechnen (Lazy computation) 8
3.2 Graphen 8
3.3 Constructive Solid Geometry (CSG) 8
3.4 Grafische-Programmierung 9
4 Mein Lösungsansatz **10**
4.1 Abhängigkeits-Graph 10
4.2 Framework 10
4.3 Graphischer Editor 11
4.4 Template 11
4.5 Generator 13
4.6 Implementation (?besser: Implementierung, weil sie den Prozess anstatt das Ergebnis meint?) 17
4.7 Beispiele 19
5 Analyse **21**
5.1 Theoretische Laufzeit 21
5.2 Reduktion der Neuberechnungszeit 22
5.3 Overhead 23
5.4 Nutzerfreundlichkeit 23
5.5 Erweiterbarkeit 24
6 Future Work **25**
6.1 Nullwerte führen zu leeren Lösungen 25
6.2 Andere Datenstrukturen 25
6.3 Nutzerfreundlichkeitsstudie 25
6.4 Parallelisierung der Generierung 25
6.5 Automatische Cache-Optimierung 26
6.6 Skalierung auf sehr große Welten 26
6.7 Integration in bestehende Engines 26
6.8 Serialisierung und Persistenz 26
7 Fazit **27**
8 Bibliografie **28**
9 Eigenständigkeitserklärungen **30**
9.1 Nutzung KI basierte Anwendungen 33

2 Stand der Technik

Dieses Kapitel gibt einen Überblick über bestehende Ansätze zur prozeduralen Generierung und bewertet, inwieweit sich diese zur minimalen Neuberechnung eignen. Zunächst werden Noise-basierte Verfahren betrachtet. Diese sind in der Praxis weit verbreitet, bieten aber kaum Möglichkeiten zur gezielten Teilneuberechnung. Danach werden L-Systems und Graph-basierte Ansätze untersucht, die durch ihre explizite Regelstruktur besser für eine solche Analyse geeignet wären, es jedoch keine konkreten Ansätze gibt. Anschließend werden KI-basierte Verfahren diskutiert und warum diese für minimale Neuberechnung grundsätzlich nicht geeignet sind. Schließlich werden die Systeme von Houdini und Blender Geometry Nodes erklärt, welche den gleichen Ansatz wie diese Arbeit verfolgen, aber für einen anderen Kontext ausgelegt sind.

2.1 Noise & Zufälligkeit

Viele prozedurale Generatoren nutzen Noise-Algorithmen, um Welten zu erzeugen. Die Grundidee ist, mehrere Noise-Funktionen wie Perlin-Noise oder Simplex-Noise mit unterschiedlichen Frequenzen und Amplituden zu überlagern. Ein Berg entsteht zum Beispiel durch eine Noise-Funktion mit niedriger Frequenz und hoher Amplitude, während kleinere Details wie Steine durch eine Funktion mit hoher Frequenz und niedriger Amplitude hinzugefügt werden. Die Struktur der Welt ergibt sich also nicht aus expliziten Regeln, sondern aus dem Zusammenspiel dieser Funktionen.

Der praktische Vorteil ist, dass sich dieser Ansatz leicht implementieren lässt und gut auf große Welten skaliert. Minecraft nutzt zum Beispiel Perlin-Noise, um sein Gelände und Höhlen effizient zu generieren.

Das Problem ist, dass sich mit Noise-Funktionen nur schwer das Einhalten global geltender Regeln garantieren lässt. Ob ein generiertes Gelände zum Beispiel immer einen zugänglichen Weg zwischen zwei Punkten hat, lässt sich aus den Noise-Funktionen allein nicht bestimmen. Solche Regeln müssen in späteren Generationsschritten überprüft und die Welt gegebenenfalls angepasst werden. Jedoch werden in vielen Generationssystemen diese Fehler toleriert, wenn sie nur selten auftauchen.

Für minimale Neuberechnung sind Noise-basierte Verfahren besonders ungeeignet, da theoretische jede Noise-Ebene Einfluss auf jedes Ergebnis hat. Die Generierungslogik steckt implizit in der Komposition der Funktionen. Ändert man eine Funktion oder ihre Parameter, kann dies zu einer komplett anderen Welt führen.

2.2 Example based Model Synthesis

Eine bekannte Arbeit im Bereich der constraint-basierten Generation ist Paul Merrels „Example-based Model Synthesis“. [1]

Die Grundidee ist hier, aus einem kleinen Eingabedatensatz eine größere Struktur zu erzeugen, die lokal denselben Regeln folgt.

Der Eingabedatensatz besteht aus einer Gitterstruktur, in der jeder Zelle ein Wert zugeordnet ist. Aus diesem Gitter wird eine Liste aller Nachbarkombinationen erstellt, die im Eingabedatensatz vorkommen.

Diese Liste enthält die Regeln, nach denen ein neues Modell erzeugt wird. Der Generationsalgorithmus läuft dabei wie folgt ab: Zunächst werden allen Zellen des neuen Gitters sämtliche möglichen Werte zugeordnet. Dann wird die Zelle mit den wenigsten verbleibenden Möglichkeiten ausgewählt und auf einen einzelnen Wert festgelegt. Anschließend werden aus den Nachbarzellen alle Werte entfernt, für die keine gültige Regel mehr existiert. Dieser Bereinigungsschritt wird rekursiv auf alle betroffenen Nachbarzellen ausgeweitet. Der Vorgang wiederholt sich, bis jede Zelle genau einen Wert enthält.

2.2.1 Minimale Neuberechnung mit Model Systhesis

Ein möglicher Ansatz zur partiellen Neuberechnung prozedural generierter Welten könnte auf Verfahren der Example-based Model Synthesis aufbauen. Die grundlegende Idee wäre, zunächst eine vollständige Welt mit dem Model-Synthesis-Algorithmus zu generieren. Wenn sich anschließend die zugrunde liegenden Regeln ändern, könnte versucht werden, nur die Teile der Welt zu verändern, die den neuen Regeln nicht mehr entsprechen.

Ein naheliegender Ansatz wäre, alle Felder zu identifizieren, deren aktuelle Werte gegen die neuen Regeln verstoßen. Für diese Felder müssten anschließend wieder mehrere mögliche Werte zugelassen werden, sodass der Model-Synthesis-Algorithmus erneut eine konsistente Lösung finden kann.

Dabei zeigt sich jedoch ein grundlegendes Problem: Wenn einem Feld neue mögliche Werte hinzugefügt werden, sind diese zunächst nicht notwendigerweise mit den aktuellen Werten der Nachbarfelder kompatibel. Damit die lokalen Regeln wieder erfüllt sind, müssten auch den Nachbarfeldern zusätzliche mögliche Werte hinzugefügt werden. Dieser Prozess kann sich wiederum auf deren Nachbarn ausbreiten und so weiter.

Würde man dieses Verfahren naiv implementieren, indem für alle betroffenen Nachbarn wieder sämtliche möglichen Werte zugelassen werden, entstünde im Extremfall erneut ein vollständig unentschiedenes Gitter. In diesem Fall würde der Algorithmus effektiv wieder bei einem normalen Model-Synthesis-Prozess beginnen, wodurch kein Vorteil gegenüber einer vollständigen Neugenerierung entsteht.

Um dennoch einen Nutzen aus diesem Ansatz zu ziehen, müsste man eine möglichst kleine Menge an Feldern finden, deren Wertebereiche erweitert werden, sodass anschließend wieder eine konsistente Lösung existiert. Diese Menge sollte idealerweise minimal sein, damit möglichst große Teile der bestehenden Welt unverändert bleiben können. Gleichzeitig müsste der Aufwand zur Bestimmung dieser Menge deutlich geringer sein als eine komplette Neugenerierung der Welt.

Eine theoretische Möglichkeit bestünde darin, den Raum der möglichen Werteerweiterungen systematisch zu durchsuchen. Beispielsweise könnte eine Breitensuche über den Graphen der möglichen Wertkombinationen durchgeführt werden, um eine minimale

Menge an Änderungen zu finden, die wieder zu einer gültigen Konfiguration führt. Allerdings wächst dieser Suchraum sehr schnell und führt sowohl in Bezug auf Laufzeit als auch Speicherverbrauch zu erheblichen Komplexitätsproblemen.

Der Vorteil des ursprünglichen Model-Synthesis-Algorithmus liegt darin, dass zu jedem Zeitpunkt alle noch möglichen Kombinationen eine valide Lösung darstellen. Das Finden einer minimalen Erweiterung dieser Mengen, die nach einer Regeländerung wieder eine gültige Lösung ermöglicht, ist jedoch deutlich schwieriger als die ursprüngliche Generierung selbst.

2.3 Graph Grammatiken

Eine Graph Grammatik ist ein System aus Regeln, die beschreiben, wie ein Graph verändert werden kann. Jede Regel besteht aus zwei Teilen: einem Teilgraph, der gesucht wird, und einem Teilgraph, der ihn ersetzt. Durch wiederholtes Anwenden solcher Regeln kann aus einem kleinen Startgraph eine komplexe Struktur wachsen. Zum Beispiel kann aus einem einzelnen Knoten durch eine Regel wie „Füge zwei Kindknoten hinzu„ ein ganzer Baum entstehen.

2.3.1 L-Systems

L-Systems sind eine frühe Anwendung dieser Idee, entwickelt in den 1960ern vom Biologen Aristid Lindenmayer, um Pflanzenwachstum zu modellieren. Statt auf einem Graphen arbeiten sie auf einer Sequenz von Symbolen. In jedem Schritt werden alle Symbole gleichzeitig durch die passende Regel ersetzt. Ein Startsymbol F könnte die Regel haben „ersetze F durch F[+F][-F]„, wobei + und - für eine Drehung stehen. Nach wenigen Iterationen entsteht so eine verzweigte Struktur, die wie ein Baum aussieht.

L-Systems eignen sich gut für Vegetation, weil natürliche Strukturen oft selbstähnlich sind. Ein Ast sieht aus wie ein kleiner Baum, ein Zweig wie ein kleiner Ast. Mit wenigen Regeln lassen sich so sehr organisch wirkende Formen erzeugen.

2.3.2 Graph-based Model Synthesis

Graph-based Model Synthesis erweitert das Konzept von Graph Grammatiken, indem Regeln automatisiert aus einem Beispiel abgeleitet werden. Das heißt, dass ein bestehendes Modell in kleine Strukturelemente zerlegt wird. Daraus werden Regeln abgeleitet, wie Teilgraphen ersetzt werden dürfen. Durch wiederholtes Anwenden dieser Regeln entstehen neue Modelle, die lokal die gleichen Regeln des Beispiels folgen, aber global andere Strukturen haben können. Im Vergleich zu L-Systems können Regeln hier auf beliebige Graphstrukturen verweisen und komplexere räumliche Bedingungen beschreiben. Außerdem ist das Verfahren nicht auf lineare Symbolsequenzen beschränkt, wodurch es sich besser für dreidimensionale Strukturen eignet.

2.3.3 Minimale Neuberechnung

Keiner dieser Ansätze unterstützt minimale Neuberechnung. Bei L-Systems baut jede Iteration direkt auf der vorherigen auf. Eine Regeländerung macht damit alle folgenden Iterationen ungültig, unabhängig davon, wie klein die Änderung ist. Bei Graph-based

Model Synthesis ist das Problem ein anderes: Die Regeln werden aus einem Beispiel abgeleitet und beschreiben lokale Nachbarschaftsbedingungen. Ändert sich eine Regel, ist nicht klar welche Teile des generierten Modells diese Bedingung noch erfüllen, ohne die gesamte Struktur neu zu prüfen. In beiden Fällen fehlt eine explizite Darstellung, welche Teile des Ergebnisses von welchen Regeln abhängen. Genau das ist jedoch die Voraussetzung für minimale Neuberechnung.

2.4 KI basierte prozedurale Generation

Neuronale Netzwerke können genutzt werden, um aus einen Trainingsdatensatz, der eine Menge an Beispielen enthält, ein weiteres ähnliches Beispiel zu erzeugen. In Arbeiten wie [2], [3] und [4] werden Neuronale Modelle genutzt, um Levels aus bekannten 2D-Spielen zu reproduzieren. Dabei wird in [2] ein Level-Generations-Model gegen ein Spieler-Agent-Modell, welches die „Spielbarkeit“ eines Levels bewertet, trainiert. Dieser Ansatz ist in dem meisten Fällen in der Lage, spielbare Levels zu generieren. Vereinzelt werden jedoch Generationsartefakte, zum Beispiel unerreichbare Plattformen oder auch komplett leere Bereiche, erzeugt. Dazu muss gesagt werden, dass die oben genannten Paper „nur“ die simplen und zahlreich vorhandenen 2D-Welten des Spiels „Super Mario“ als Trainingsdatensätze heranziehen. Offen ist, ob der vorgestellte Ansatz auch in neueren und komplexeren Spielen funktioniert.

2.4.1 Terrain Diffusion

Im Ansatz „Terrain Diffusion“ werden Neuronale-Diffusion-Modelle verwendet. Diffusion-Modelle erzeugen Daten, indem sie iterativ aus zufälligem Rauschen eine strukturierte Lösung rekonstruieren. Dieses Verfahren wurde ursprünglich für die Bildgenerierung entwickelt. Heute wird es zum Beispiel auch für das Erzeugen von Höhenkarten genutzt.

Hier wird ein Neuronales Modell mit realen Geländedaten trainiert, damit es realistische Terrainstrukturen erzeugen kann. Um auch weiterhin unendliche Welten generieren zu können, wird das Gelände in Form von Kacheln erzeugt, die deterministisch aus einem sogenannten „Seed“ generiert werden. So können, ähnlich wie bei klassischen Noise-basierten Verfahren, bei Bedarf neue Bereiche der Welt errechnet werden.

Der Vorteil dieses Ansatzes liegt darin, dass größere geografische Strukturen realistischer modelliert werden können als mit klassischen Noise-Funktionen.

Ein Nachteil ist jedoch, dass das Verhalten des Systems stark vom Trainingsdatensatz abhängt und Generationsregeln nicht explizit definiert sind. Dadurch ist es schwierig sicherzustellen, dass bestimmte strukturelle Anforderungen immer erfüllt werden. [5]

2.4.2 Minimale Neuberechnung

KI-basierte Ansätze teilen ein grundlegendes Problem: Die internen Repräsentationen neuronaler Modelle sind nicht interpretierbar. Es kann also nicht nachvollzogen werden, welche Teile der Eingabe oder welche Gewichte des Modells in einer bestimmten Weise zum generierten Ergebnis beitragen. Ändert sich ein Parameter oder eine Eingabe, kann man daraus nicht ableiten, welche Teile einer bereits generierten Welt noch gültig sind.

Die minimale Neuberechnung erfordert jedoch genau diese Nachvollziehbarkeit. Aus diesem Grund werden KI-basierte Ansätze in dieser Arbeit nicht weiter betrachtet.

2.5 Houdini & Blender Geometry Nodes

Programme wie Houdini und Blender Geometry Nodes nutzen Abhängigkeits-Graph-Systeme, wie sie in dieser Arbeit vorgestellt werden. Dabei werden auch Zwischenergebnisse gespeichert und wiederverwendet, um die Laufzeit zu verbessern. Beide Systeme sind jedoch auf die interaktive Erstellung einzelner Assets ausgelegt und nicht auf die prozedurale Generierung großer, zusammenhängender Welten mit Tausenden von Elementen. Zudem sind sie geschlossene Programme, die nicht darauf ausgelegt sind, in eine Spiel- oder Simulations-Engine eingebettet zu werden. Das in dieser Arbeit vorgestellte System ist aber genau dafür konzipiert. Der Generationsalgorithmus soll in der Umgebung iterativ weiterentwickelt werden können und Änderungen in einer bereits bestehenden Welt sollen unmittelbar sichtbar werden.

3 Theoretische Grundlagen

Dieses Kapitel beschreibt die technischen Konzepte, auf denen das in dieser Arbeit vorgestellte System aufbaut. Dazu gehören Lazy Computation als Grundprinzip der minimalen Neuberechnung, Graphen als zentrale Datenstruktur, Constructive Solid Geometry zur Darstellung von Geometrie sowie Grafische Programmierung als Grundlage des Editors.

3.1 Minimales Berechnen (Lazy computation)

Minimales Berechnen (englisch: lazy computation) beschreibt die Idee in Computerprogrammen die Daten erst dann zu berechnen, wenn sie benötigt werden.

Es reduziert Lag-spikes gerade beim starten eines neuen Prozesses, da Anstatt alle nutzbaren Daten nur die, die gerade angefragt benötigt werden errechnet werden.

In Funktionale Programmiersprachen wie Haskell findet dieses Konzept vie Anwendung und erlaubt für unendliche Datenstrukturen.

3.2 Graphen

Die Datenstrukturen in dieser Arbeit basieren auf mathematische Graphen.

Daher erklärt dieser Abschnitt die in der weiteren Arbeit genutzten Notationen.

Ein Graph $G := (V, E)$ ist besteht aus einer Menge an Knoten V und einer Menge an Kanten E. Die Funktionen $V(G) = V$ und $E(G) = E$ werden als verkürzte Notationen genutzt.

Ein einzelner Knoten wird als $v \in V$ und eine einzelne Kante als $e \in E$ geschrieben.

Eine Kante ist ein Tuple zweier Knoten $e := (v_a, v_b)$. In dieser Arbeit sind Kanten grundsätzlich gerichtet und werden dann auch als $\overrightarrow{v_a v_b}$ geschrieben.

Die Menge aller eingehenden bzw. ausgehender Kanten eines Knoten v wird als $E_G^-(v)$ und $E_G^+(v)$ geschrieben.

Die Menge aller Knoten die eine eingehende bzw. ausgehende Kante zu einem Knoten haben werden als $N_G^-(v)$ und $N_G^+(v)$ geschrieben, da sie als die Nachbarn von diesem Knoten verstanden werden.

Diese Notationen basieren auf den Notationen in „Modern Graph Theory“ von Béla Bollobás [6].

3.3 Constructive Solid Geometry (CSG)

Constructive Solid Geometry (CSG) ist eine Methode zur Darstellung von Volumen, bei der komplexe Geometrien durch die Kombination primitiver Geometry (Box, Kugel, etc) dargestellt wird.

Diese Kombination sind standartmäßig Vereinigung, Schnittmenge und Differenz, wurden jedoch in späteren Arbeiten mit komplexeren Operationen, wie Verformung, Duplikation, etc erweitert.

CSGs werden als directed acyclic graph (DAG) gespeichert. Jede Knoten is eine implizierte Operation auf den Volumen der Kinder. Die Blätter sind dahingegen durch Parameter definierte primitive Geometrien.

Der Vorteil dieser Darstellung ist, dass Geometrie über Parameter und Operationen beschrieben wird. Änderungen an Position, Größe oder anderen Parametern wirken sich direkt auf das resultierende Volumen aus, ohne dass die gesamte Geometrie neu definiert werden muss.

Jedoch eignen sich CSGs selten zum direkten rendering mit raytracing, da die Operationen zu leistungsaufwendig sind um mit jeden Ray den DAG zu rekursiv zu iterieren. Dazu steigt die Größe des DAGs schnell mit der Komplexität des CSGs. [7], [8]

3.4 Grafische-Programmierung

Grafische-Programmierung ist eine From der Programmierung bei der Logik nicht als Code definiert ist sonder als Diagram in einem grafischen Editor. Funktionen werden als Knoten mit Input und Output dargestellt. Dies können mit Linien verbunden werden um logische Abfolgen zu definieren.

Grafische-Programmierung erlaubt ein besseren überblick über Programmabschnitte und ist durch seinen invitieren Syntax Einsteigerfreundlicher.

Grafische-Programme ausgeführt werden indem die Operationen als Graph dargestellt werden. Dieser kann vergleichbar zu Funktionenprogrammiersparchen rekursiv gelöst werden.

Jedoch ist es auch möglich einen Grafische-Programme in Code zu übersetzen und Maschinen-Code zu kompilieren. Dies kann massive Leistungsvorteile haben.

Populäre Programme in den Grafische-Programmierung verwendet wird, sind Unity Shader Graphs, Unreal Engine Templates und Blender Geometry Nodes.

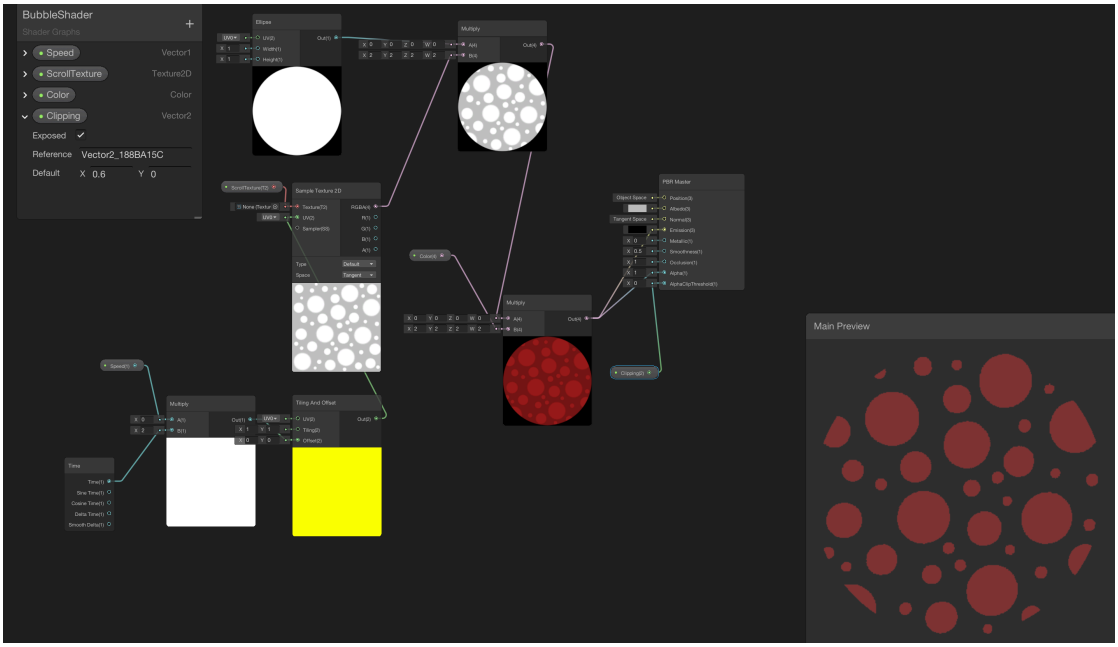

Abbildung 1: Der grafischen Editor von Unity Shader Graph

4 Mein Lösungsansatz

Dieses Kapitel beschreibt, wie das Ziel, die Neuberechnungszeit zu verkürzen, erreicht werden kann. Die Komponenten des Systems werden vorgestellt und zentrale Algorithmen werden erklärt. Weiterhin werden Besonderheiten im System sowie mögliche Probleme diskutiert.

Mein Lösungsansatz beschreibt, wie prozedurale Generierung strukturiert sein sollte, damit das System automatisch erkennt, welche Teile der Welt es bei einer Änderungen im Algorithmus neu berechnen muss. Dafür wird der Algorithmus als Abhängigkeitsgraph modelliert.

4.1 Abhängigkeits-Graph

Jede mathematische Formel oder jeder Algorithmus kann als Graph an Abhängigkeiten dargestellt werden. Dabei beschreibt jeder Knoten eine mathematische Operation, die aus einem oder mehreren Eingangswerten ein Ergebnis errechnet. Dabei gilt, dass als Eingangswerte für Knoten die Ergebnisse anderer Knoten im Graph verwendet werden. Dies bezeichne ich als Abhängigkeit, weil für die Berechnung eines Knotens alle Ergebnisse, die hierfür Eingangswerte sind, zuvor errechnet werden müssen. Das Ergebnis eines Knotens hängt nur von seinen Eingangswerten ab und hat damit keine Nebeneffekte (siehe Funktionale Programmierung). Knoten und deren Ergebnisse, die keine Eingangswerte haben, bezeichne ich als konstant.

4.2 Framework

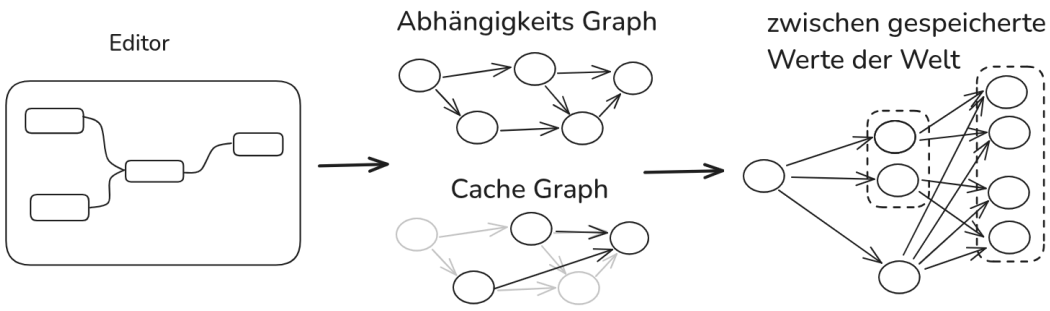

Abbildung 2: Überblick über Editor, Abhängigkeits-Graph, dessen cache und der Welt.

TODO: Sauber zeichnen

Mein Generationssystem besteht aus drei Bestandteilen:

1. Ein graphischer Editor, mit dem ein Nutzer einen Abhängigkeits-Graph erstellen und bearbeiten kann.
2. Das Template ist eine Datenstruktur, die vom Editor erstellt wird. Es enthält den Abhängigkeits-Graph sowie Anleitungen wie dieser generiert und zwischengespeichert werden soll.

3. Der Generator vergleicht das aktuelle Template mit dem neuen Template und generiert jene Bestandteile der Welt neu, die nicht dem neuen Template entsprechen.

4.3 Graphischer Editor

Zur interaktiven Definition des Abhängigkeits-Graph wird ein grafischer Programmierungseditor, vergleichbar mit Unreal Templates, Blender Geometry Nodes oder Unity Shader Graph, genutzt.

Der Nutzer kann Knoten erstellen, die einer Operation entsprechen, und diese auf einer unendlichen Fläche frei anordnen. Diese Operationen haben auf ihrer linken Seite eine Liste mit Eingangswerten und auf ihrer rechten Seite eine Liste mit Ergebnissen.

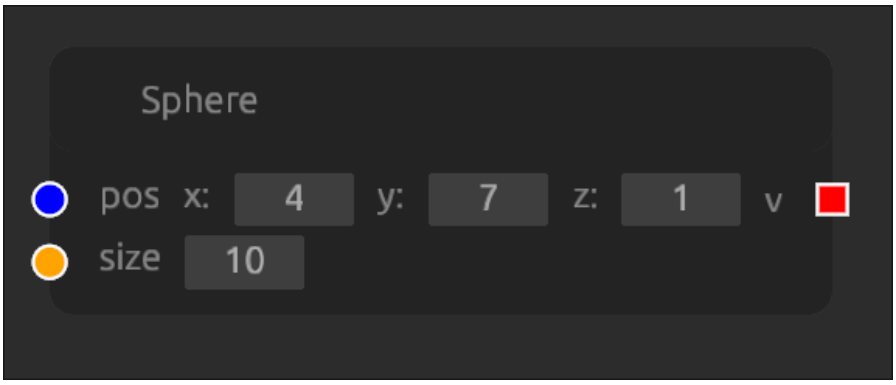

Abbildung 3: Node um ein Kugel Volumen zu definieren. **TODO:** Hintergrund

Die Eingangswerte und Ergebnisse sind je nach Datentyp farbig kodiert und können mit Linien verbunden werden. Dies zeigt, dass ein Ergebnis als Eingangswert für eine andere Operation verwendet werden soll.

Um komplexe Algorithmen zu modellieren, werden Knotenverbindungen links nach rechts aufgereiht. Parallele Stränge werden übereinander angeordnet. Damit werden auch komplexe Abhängigkeiten übersichtlich dargestellt.

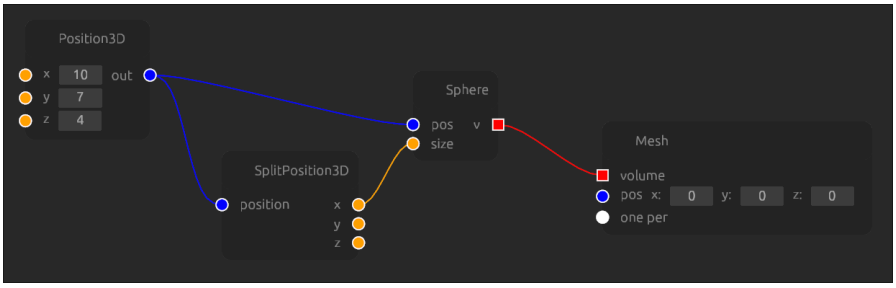

Abbildung 4: Eine Kugel dessen Größe seiner X Position entspricht. **TODO:** Hintergrund

4.4 Template

Das Template $:= (G_{\text{ab}}, G_{\text{ch}})$ besteht aus dem Abhängigkeits-Graphen G_{ab} und einem Cache-Graphen G_{ch}.

G_{ab} ist ein Graph, der die zu generierende Welt als rekursive Formel beschreibt, die aus den oben genannten Operationen besteht.

Die Eingehenden Nachbarn $N^-_{G_{\text{ab}}}$ errechnen die Eingangswerte für eine Operation und die Ausgehenden Nachbarn $N^+_{G_{\text{ab}}}$ sind alle Operationen, die das Ergebnis benötigen.

ToDo:

Beispiel?

G_{ch} enthält einen Knoten für jeden Knoten in G_{ab}, der zwischengespeichert werden soll. Dies ist eine Untermenge aller Knoten in G_{ab} $V(G_{\text{ch}}) \subset V(G_{\text{ab}})$.

Bei der Entscheidung, wie groß diese Untermenge sein soll, müssen der „Overhead“ durch Zwischenspeicherung und die Zeitersparniss durch Wiederverwendung der Ergebnisse abgewägt werden.

Die Eingehenden Nachbarn $N^-_{G_{\text{ch}}}$ sind alle Caches, von denen der Knoten abhängt. Man findet diese, indem man den Baum der Abhängigkeiten in G_{ab} in allen seinen Verzweigungen rekursiv durchsucht, bis man jeweils auf einen Knoten in G_{ch} stößt.

ToDo:

Grafik von Datentypen und Abhängigkeits-Graph

4.4.1 Level

Wenn der G_{ab} ein Directed Acyclic Graph (DAG) ist, kann jedem Knoten $v \in G_{\text{ab}}$ ein Level $l(v)$ zugeordnet werden. Dies ist definiert als:

$$l(v) > l(v_i) \quad \forall v_i \in N^-_{G_{\text{ab}}}(v)$$

Somit ist das Level eines Knotens immer größer als das Level aller Knoten von denen er abhängt.

Um den G_{ab} zu errechnen, werden die Knoten der Level aufsteigend errechnet. Innerhalb eines Levels hat die Reihenfolge keine Auswirkung.

4.4.2 Einen prozeduralen Algorithmus als Template darstellen

Bei der Implementierung habe ich mich entscheiden, dass das finale Ergebnis des Templates ein Volumen als „constructive solid geometry“ (CSG) sein soll.

Diese CSG setzt sich aus Remove- und Union-Operationen auf primitiven geometrischen Körper wie Kugeln und Boxen zusammen.

Einige der Operationen, die ich implementiert habe, sind:

- Kugel aus Position und Durchmesser
- Box aus Position und Seitenlänge
- alle Positionen auf einem Gitter, die innerhalb eines Volumens sind.
- eine Menge an zufälligen Positionen innerhalb eines Volumens.
- Addition, Subtraktion, Multiplikation und Division von Positionen und Zahlen

4.4.3 Generation eines Templates

ToDo:

Dieses Kapitel solltest du neu aufbauen und die Aussagen in eine nachvollziehbare Reihenfolge bringen: Das Wichtigste zuerst, dann Einzelheiten und Schritte usw., dann Besonderheiten

Operationen im Abhängigkeits-Graph G_{ab} können auch Mengen an Werten erzeugen. In meiner Implementierung sind dies die beiden Operationen, die einerseits ein Gitter und andererseits zufällige Positionen in einem Volumen errechnen.

Es kann vorkommen, dass die weiteren Operationen nicht auf der ganzen Menge, sondern pro Element ausgeführt werden sollen. Dieser Schritt im prozeduralen Algorithmus ermöglicht es, iterativ immer feiner werdende Details zu generieren.

Zu Beispiel errechnet man eine Menge an Positionen an den Apfelbäume stehen sollen und dann generiert man pro Baum die Positionen der Äpfel.

ToDo:

ganzen Absatz nochmal prüfen, Beispiel im Zweifel weglassen

In meinem System arbeiten alle Algorithmen nur auf den Knoten des Templates. (Begründung ergänzen: …)

Dies führt zu einem klaren Unterschied in der Laufzeit von Algorithmen auf dem Template im Vergleich zu Algorithmen auf der generierten Welt. Die Menge an Knoten im Abhängigkeits-Graphen und so auch im Cache-Graphen skaliert mit der Menge an Operationen des Generationsalgorithums. Wobei die Menge der Elemente in der generierten Welt mit den Größen der Mengen an rechneten Werten skaliert. In anderen Worten: Alle Knoten im Template zu iterieren, ist relativ schnell möglich. Hingegen kann die Laufzeit exponentiell ansteigen, wenn alle Elemente in der Welt zu iterieren sind.

Sie nutzen die Abhängigkeiten im Template, um herauszufinden wie die Welt neu generiert werden muss.

4.5 Generator

Der Generator enthält einen Graphen G_{gen}, der dem Cache-Graphen G_{ch} im Template entspricht. Jeder Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ speichert, welchem Knoten $v_{\text{ch}} \in V(G_{\text{ch}})$ er entspricht $v_{\text{ch}} = \textbf{cache}(v_{\text{gen}})$. Dazu hat ein Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ das (?gleiche?) Level wie sein Cache-Template-Knoten $l(v_{\text{gen}}) = l(\textbf{cache}(v_{\text{gen}}))$.

Jedoch wenn G_{ch} nur einen Knoten pro Operation enthält, enthält G_{gen} einen Knoten pro Ergebnis, welches errechnet werden muss.

ToDo:

Beispiel

Dafür enthält der Generator $:= (G_{\text{gen}}, Q_{\text{tasks}})$ eine Queue Q_{tasks}, die zwei Arten von Aufträgen auf G_{gen} nach ihren Levels sortiert.

$$\text{pop}(Q_{\text{tasks}}) := \min_{q \in Q_{\text{tasks}}} (l(q))$$

Berechnungs-Aufträge ermitteln das Ergebnis eines Knoten in G_{gen}. Kind-Update-Aufträge erzeugen oder löschen Kinder (?Wort?), bis ihre Anzahl für das Template geeignet ist.

4.5.1 Abhängigkeiten-Werte im Generator-Graph finden

Um einen Knoten in G_{gen} zu errechnen, benötigt man die Ergebnisse der Knoten in G_{gen}, von denen dieser Knoten abhängt. Wie in Abschnitt 4.4.3 erläutert, ist es innerhalb einer vertretbaren Laufzeit nicht möglich, diese zum Beispiel mit einer Tiefensuche zu finden.

Stattdessen wird für jeden Template-Knoten $v \in V(G_{\text{ch}})$ einer der Knoten, von den dieser abhängt, $N^-_{G_{\text{ch}}}(v)$ als Erstellungsknoten $v_c \in V(G_{\text{ch}})$ im Template markiert $v_c = \textbf{create}(v)$.

Um nun für einen Knoten $v_{\text{gen}} \in V\big(G_{\text{gen}}\big)$ alle weitern Knoten zu finden, von dieser abhängt $N^-_{G_{\text{gen}}}\big(v_{\text{gen}}\big)$, werden die relativen Schritte in G_{ch} ausgehend vom Erstellungsknoten v_c hin zu den weiteren abhängigen Knoten als Baum gespeichert $T_{\text{rel}}\big(v_{\text{gen}}\big)$.

Ein relativer Schritt v_{step} gibt an, dass man beginnend bei einem Knoten $v \in V(G_{\text{ch}})$ entweder hoch (?aufwärts?) ($\textbf{up}(v_{\text{step}}) = \text{True}$) in einen Knoten $v_{\text{up}} \in V(G_{\text{ch}})$ gehen soll, von dem v abhängt ($v_{\text{up}} \in N^-_{G_{\text{ch}}}(v)$), oder runter (?abwärts?) ($\textbf{up}(v_{\text{step}}) = \text{False}$) in einen Knoten $v_{\text{down}} \in V(G_{\text{ch}})$, der von v abhängt ($v_{\text{down}} \in N^+_{G_{\text{ch}}}(v)$).

Da ein Knoten $v \in V(g_{\text{ch}})$ mehr als einen eingehenden oder ausgehenden Nachbarn haben kann, speichert ein relativer Schritt auch, in welchen Nachbarn gegangen werden soll ($\textbf{cache}(v_{\text{step}})$). Ein relativer Schritt Speicher weiterhin, ob dieser Nachbar eine Abhängigkeit für v_{gen} ist ($\textbf{deps}(v_{\text{step}}) = \text{True}$).

Jeder relative Schritt v_{step} speichert auch, in welche der Nachbarknoten gegangen werden soll.

Diese relativen Schritte nutzen (?berücksichtigen?) nur Knoten, die ein kleineres Level als v_{gen} haben. Da im Generator Knoten im Level in aufsteigender Reihenfolge erstellt werden, ist so sichergestellt, dass alle relativen Wege existieren.

Für einen Knoten im Template kann es mehrere Knoten im Generator geben. Daher können dort pro Abhängigkeit eines Cache-Knoten auch mehrere Knoten gefunden werden.

Algorithm 1: Finde abhänige Knoten in G_{gen}

```
procedure FindDeps(v_gen, v_gen creates)
    T ← T_rel(v_gen)
    D ← ∅
    Q ← ∅
    push(Q , (v_root ∈ T, v_gen creates))

    while Q ≠ ∅ do
        (v_step, v_gen) ← pop(Q)

        if deps(v_step) then
            push(D , v_gen)
        end

        for v_child step ∈ N_T^+(v_step) do
            N ← N_{G_gen}^-(v_gen) if up(v_child step) else N_{G_gen}^+(v_gen)

            for v ∈ N do
                if cache(v_child step) = cache(v) then
                    push(Q, (v_child step, v))
                end
            end
        end
    end
    return D
end
```

ToDo:

Beispiel Zeichung

4.5.2 Kind-Update-Aufträge

Kind-Update-Aufträge enthalten den Index des Erstellungsknoten und den Index eines Erstellungseintrags $E_{\text{create}}(v_{\text{ch}})$ in dessen Template-Knoten (?gehört „in dessen Template-Knoten“ hierher oder eher direkt hinter „Kind-Update-Aufträge?).

Dieser Erstellungseintrag definiert, wie viele Kinder es geben soll $\textbf{num}(v_{\text{ch}}, v_{\text{gen creates}})$. Dies sind entweder genau n pro Erstellungsknoten oder hängen von dem Wert des Erstellungsknotens $v_{\text{gen creates}}$ ab, wie z.B. einer Positionsmenge. Dazu gibt **valid** $(v_{\text{gen}}, v_{\text{gen creates}})$ an, ob ein Kind v_{gen} für den Erstellungsknoten $v_{\text{gen creates}}$ noch valide ist, also ob beispielsweise eine Position noch in der Menge an Positionen ist.

Daraufhin wird die vorhandene Menge an Kindern mit der gewünschten Menge verglichen. Bei Ungleichheit werden neue Kinderknoten (?oder muss es hier auch „Kinder“ heißen wie oben im Kap. Generator?) erzeugt oder gelöscht.

Wenn eine neuer Knoten erzeugt wird, werden mit dem Baum an relativen Schritten die Indizes aller abhängige Knoten gesucht und im Knoten gespeichert.

Algorithm 2: Kinder updaten

```
procedure UpdateChild(v_gen, v_ch child)

    C ← { v ∈ N⁺_{G_gen}(v_gen) | cache(v) = v_ch child }

    C_to delete ← { v ∈ C | ¬ valid(v, v_gen) }

    for v ∈ C_to delete do

        Delete(v)
    end

    n ← num(v_ch child, v_gen)
    i ← 0
    for i < n − |C \ C_to delete| do

        Create(v_ch child, v_gen)
    end
end

procedure Create(v_ch, v_gen creates)
    push(V(G_gen), v_new)

    cache(v_new) ← v_ch

    D ← FindDeps(v_new, v_gen creates)

    N⁻_{G_gen}(v_new) ← D

    for v ∈ D do
        push(N⁺_{G_gen}(v), v_new)
    end

    pushCalculate(Q_task, v_new)
end

procedure Delete(v_gen)

    for v ∈ N⁻_{G_gen}(v_gen) do

        remove(N⁺_{G_gen}(v), v_gen)
    end

    for v ∈ N⁺_{G_gen}(v_gen) do

        Delete(v)
    end
    remove(V(G_gen), v_gen)
end
```

4.5.3 Errechnungs Aufträge

Errechnungs Aufträge (?Berechnungsaufträge?) ermitteln den Wert eines Knotens $v_{\text{gen}} \in V(G_{\text{gen}})$ neu. Dabei wird der Knoten im Abhängigkeits-Graph rekursiv errechnet.

Wenn der Algorithmus auf einen Knoten $v_{\text{ab}} \in V(G_{\text{ab}})$ stößt, welcher einen Cache-Knoten hat $v_{\text{ab}} \in V(G_{\text{ch}})$, werden die Werte der jeweiligen abhängigen Knoten von v_{gen} verwendet.

ToDo:

Beispiel

4.5.4 Abhängigkeitskreise

Das Template kann Abhängigkeitskreise enthalten. Um dennoch valide Lösungen errechnen zu können, muss es für jeden Knoten v_{val} einen validen Nullwert geben.

So kann der Abhängigkeitsgraph iterativ gelöst werden. Pro Kreis im Abhängigkeits-Graph wird eine Kante als durchgeschnitten markiert $N^{+}_{\text{cut}}(v) \subseteq N^{+}_{G_{\text{dep}}}(v) \quad v \in V(G_{\text{dep}})$. Der Abhängigkeitsgraph ohne die durchtrennte Kanten $N^{+}_{\text{not cut}}(v) := N^{+}_{G_{\text{dep}}}(v) \setminus N^{+}_{\text{cut}}(v)$ ist ein DAG (!in Klammern einmal ausschreiben!). Folglich kann jedem Knoten ein Level $l(v)$ zu geordnet werden.

$$l(v) > l(v_i) \quad \forall v_i \in N^{+}_{\text{not cut}}(v)$$

Die Knoten werden Level für Level erzeugt. So wird sichergestellt, dass alle nicht-geschnittenen Abhängigkeiten bereits errechnet wurden, wenn der Knoten selbst errechnet wird. Hat ein Knoten geschnittene Abhängigkeiten, werden diese für das Errechnen genutzt, sofern sie existieren. Andernfalls wird der Nullwert verwendet. Jeder Knoten, der Nullwerte für seine geschnittenen Abhängigkeiten nutzt, wird erneut errechnet, sobald alle Knoten einmal errechnet wurden. Dies wird so oft wiederholt, bis keine Nullwerte mehr verwendet werden.

4.6 Implementation (?besser: Implementierung, weil sie den Prozess anstatt das Ergebnis meint?)

Für die Beispiel-Implementation (?…?) habe ich Rust als Programmiersprache gewählt, da sie erlaubt, speichersicheren Lowlevel-Code zu schreiben, um die Laufzeit von Algorithm effektiv zu verbessern. Zudem hat sie einen (im Gegensatz zu C++) umfassenden und einfach zu nutzenden Package-Manager.

4.6.1 Stabile Listen

Alle Graphen sind mit stabilen Listen implementiert.

Eine stabile Liste ist ein sich automatisch vergrößerndes Array. Wenn dass aktuelle Array voll ist wird ein größeres Array alloziert und die Werte mit einer Memcopy-Operation kopiert. Die Indizes von Elementen ändern sich in stabilen Listen nicht. Wenn ein Element entfernt wird, werden die weiteren Elemente nicht verschoben, um die Lücke zu schließen. Stattdessen wird der Index des nächsten freien Elements gespeichert. Dazu speichert die Liste den Index des ersten freien Elements, welches genutzt wird, wenn ein neues Element eingefügt wird. Der dort gespeicherte nächste freie Index wird dann als erster freier Index gespeichert. Dies erlaubt Einfüge-, Entfernungs- und Zugriffslaufzeit von $O(1)$.

Weiterhin wird pro Element eine Versionsnummer gespeichert. Bei einem Zugriff wird die Version des Indexes mit der Version des Elements an der Stelle des Index verglichen. Indizes bleiben valide, solange das Element in der Liste ist. Und es wird erkannt, wenn man mit einem veralteten Index zugreift.

Die Versionsnummer wird in den oberen 32 Bit des Indexes gespeichert. Somit können in einer stabilen List 2^{32} Elemente gespeichert werden.

Stabile Listen ermöglichen es, Graphen effizient als Listen darzustellen, indem in den Knoten die Indexe der anderen Knoten gespeichert werden, zu den Kanten existieren. Stabile listen sind dabei wesentlich schneller als HashMaps. [9]

4.6.2 Multi Threading

Der Collapser sowie die Sampling-Operationen werden in asynchronen Workern ausgeführt. Für die Kommunikation werden Channel verwendet. [10] & [11]

Der Composer läuft im Render Thread und errechnet bei jeder Änderung das aktuelle Template. Dieses wird über einen Channel zum Collapser gesendet. Wenn der Channel noch ein altes Template enthält, werden die Änderungsnotizen zum alten Template dem neuen Template hinzugefügt. Danach wird das Template wird ersetzt.

Die errechnete CSG-Darstellung der Welt wird mit einem weiteren Channel an die Sampler gesendet, welche die CSG-Darstellung in ein Voxel DAG oder Mesh umrechnen und auf die GPU transferieren.

Sobald die neue Welt hochgeladen wurde, wird der Render Thread über einen Channel über diese Änderung informiert.

4.6.3 Small Vectors

Für die Zwischenspeicherung der Werte werden Small Vectors verwendet. Diese haben die Eigenschaft, dass die ersten N Elemente direkt auf dem Stack alloziert werden. Sobald diese voll sind, wird ein Array auf dem Stack alloziert.

Alle Werte müssen hierbei als Liste behandelt werden, da ein Knoten für seinen Input immer von mehreren Knoten abhängt. Jedoch enthält diese Liste meist nur ein Element. Small Vectoren erlauben es, für diese Fälle, auf das Allozieren des Stack zu verzichten, und bieten eigene bessere Cache-Lokalitäten. [12]

4.6.4 Output Datenstruktur

Der Kernkonzept, einen sehr großen Graphen, der einen prozeduralen Algorithmus darstellt, zu bearbeiten, indem man die Abhängigkeiten in einem vergleichbaren kleineren Template nutzt, enthält keine konkreten Annahmen über die Art der Geometrie. In meiner Implementation habe ich CSG genutzt, da es einer sehr allgemeine Form ist, Volumen darzustellen, und diese zudem leicht zu bearbeiten sind.

Aber gerade CSGs mit vielen Knoten sind jedoch nicht performant zu rendern. Deshalb können sie in meiner Implementation entweder als Sparse Voxel DAGs oder mit Marching Cubes gesampled (!bitte anderes Wort!) werden.

Voxel-Datenstrukturen können relativ effizient mit Ray Marching geraytraced (!bitte anderes Wort!) werden. [13], [14]

Mit Hilfe von Marching Cubes kann ein CSG als Mesh approximiert werden. [15]

4.7 Beispiele

Im Folgenden werden mehrere Beispiele vorgestellt, die die Funktionen (?Funktionsweise?) des Systems demonstrieren. Dabei wird zuerst in minimalen Beispielen gezeigt, wie das System Teile der generierten Welt wiederverwenden kann. Danach wird eine komplexe Welt vorgestellt, welche den Umfang des Systems veranschaulicht.

4.7.1 Nur finales Volumen neu errechnen

Im ersten Beispiel wird eine große Anzahl von Bäumen generiert, die auf einem Gitter verteilt sind. Jeder Baum besteht aus einem Stamm (Zylinder) und einer Baumkrone (Kugel). Die Positionen aller Bäume werden mit einer Gitter-Operation berechnet und im Cache gespeichert. Die Form der Baumkrone wird als separate Operation im Abhängigkeitsgraphen modelliert.

Wenn nun die Form oder Größe der Baumkrone verändert wird, betrifft diese Änderung nur den Teilgraphen, der die Baumkrone berechnet. Dabei können die Positionen der Bäume wiederverwendet werden. Der Generator erkennt, dass nur die CSG-Volumen der Baumkronen ungültig sind und berechnet ausschließlich diese neu.

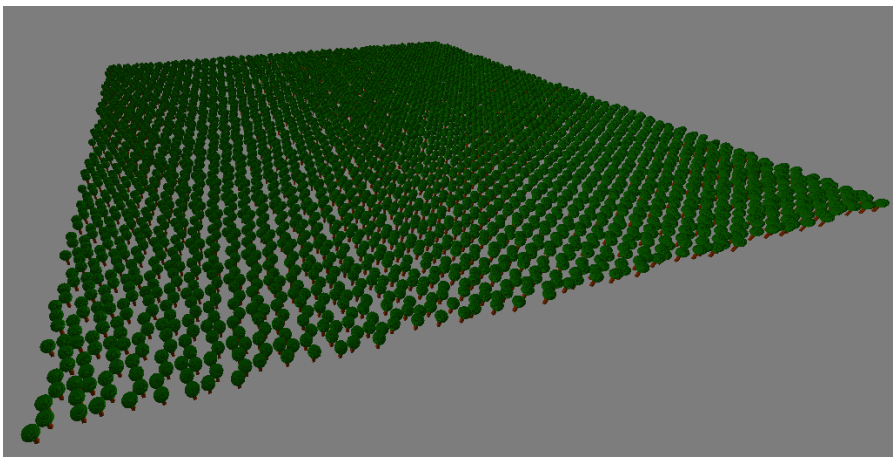

Abbildung 5: Trees

ToDo:

Update Image

4.7.2 Extern kontrollierte Variable

Knoten im Abhängigkeitsgraphen können nicht nur von anderen berechneten Knoten abhängen, sondern auch von extern kontrollierten Variablen. Ein typisches Beispiel ist die Kameraposition, die vom der Game Engine zur Verfügung gestellt wird und sich kontinuierlich verändern kann.

In diesem Beispiel wird die Kameraposition als konstanter Knoten im Abhängigkeitsgraphen modelliert. Darauf aufbauend wird eine Positionsmenge berechnet, die alle

Weltabschnitte (Chunks) innerhalb eines bestimmten Radius um die Kamera enthält. Da diese Menge direkt von der Kameraposition abhängt, wird sie neu berechnet, sobald sich die Kameraposition ändert.

Bewegt sich die Kamera, erkennt der Generator, dass der Wert des Knotens für die Kameraposition ungültig geworden ist und berechnet alle abhängigen Knoten neu. Chunks, die nun außerhalb des Radius liegen, werden gelöscht. Gleichzeitig werden für Chunks, die nun innerhalb des Radius sind, neue Knoten erzeugt und deren Werte berechnet. Chunks, die sich weiterhin im gültigen Bereich befinden, bleiben unverändert im Cache und müssen nicht neu berechnet werden.

Dieses Prinzip lässt sich über die Kameraposition hinaus auf weitere extern kontrollierte Variablen übertragen. Für ein LOD-System kann die Entfernung zur Kamera genutzt werden, um die Detailtiefe einzelner Objekte zu bestimmen. Auch für View Culling kann die Blickrichtung der Kamera als externe Variable genutzt werden, um nur sichtbare Weltbereiche zu generieren und nicht sichtbare zu verwerfen. Darüber hinaus können auch Variablen genutzt werden, die von Spielständen abhängen, um Teile der Welt dynamisch zu laden oder zu verändern, ohne den den Rest der Welt neu zu errechnen. Beispiele dafür sind, ob ein bestimmtes Gebiet betreten wurde oder ein definiertes Ereignis eingetreten ist.

Extern kontrollierte Variablen ermöglichen es daher, das System nicht nur für statische Änderungen am Generationsalgorithmus einzusetzen, sondern auch für dynamische Änderungen, auf die Spieler reagieren können.

5 Analyse

Die Bewertung eines Systems zur prozeduralen Generierung ist nicht trivial, da unterschiedliche Systeme häufig unterschiedliche Ziele verfolgen. Während einige Ansätze primär auf maximale Generationsgeschwindigkeit oder realistische Ergebnisse optimiert sind, liegt der Fokus dieser Arbeit auf der effizienten Neuberechnung nach Änderungen am Generationsalgorithmus. Ziel ist es, bei kleinen Änderungen am Generationsprozess möglichst große Teile der bereits berechneten Welt wiederverwenden zu können.

Im nächsten Abschnitt wird die theoretische Laufzeitcomplexität und des Systems analysiert.

Um meine die reale Reduktion der Neuberechnungszeit zu untersuchen wird benchmarks auf meheren Beispiel Welten durchgeführt und es wird abgeschätzt wie viel Overhead das System gegen eine minimale Implementierung eines Generationsbespiel hat.

Neben den reinen Laufzeiteigenschaften spielt auch die Nutzerfreundlichkeit des Systems eine Rolle. Durch den grafischen Editor lassen sich komplexe Abhängigkeiten oft leichter überblicken als in klassischem Quellcode. Gleichzeitig erhöht dieser Ansatz jedoch auch die Komplexität des Systems. Während klassischer Programmcode sehr flexibel ist und ohne zusätzliche Struktur auskommt, erfordert der hier vorgestellte Ansatz eine explizite Modellierung aller Abhängigkeiten im Graphen.

Im dritten sollte die Aufwand zu Erweiterung des Systems betrachtet werden. Meine Implementation nutzt nur simple Operationen auf CSGs. Dies entspricht nicht den wirklichen Datenstrukturen und Problemen in Spielen oder Simulationen. Daher würde eine Große Menge an weiteren Operationen und Datentypen implement werden müssen.

5.1 Theoretische Laufzeit

Die Kern Idee des System ist das der Generationsalgorithmus der Welt als Abhängigkeits-Graph definiert ist. Angenommen der Abhängigkeits-Graph hat a Knoten. Die errechneten Ergebnisse von manchen Knoten im Abängigkeits-Graph werden zwischen gespeichert dies ist durch den Cache-Graph definiert. Angenommen dieser hat c Knoten wobei $c \leq a$ ist. Wir definieren einen Cache-Faktor als $c_f := \frac{c}{a}$. Ein Knoten im Abhängigkeits-Graph kann mehrere Knoten im Generator haben und jeder dieser Knoten kann wieder mehrere Kinder Knoten für ein Kind im Abhängigkeits-Graph haben. Daher definieren wir einen Branche-Faktor b_f der die durchschnittliche Menge an Kindern pro Abhängigkeits-Knoten definiert. Zu letzt müssen bei einer Änderung des Abhängigkeits-Graph nur manche Knoten neu errechnet werden. Den Faktor der zu neu errechnen Knoten im Generator nennen wir g_f.

Somit ist die Laufzeitcomplexit einer Errechnung nach Template-Änderung oder Änderung einer externen Variabel:

$$O\left(\frac{\left(ac_f\right)^{b_f}}{g_f}\right) = O\left(a^{b_f}\right)$$

Dagegen ist die Laufzeit ein Template zu errechnen $O(a^2)$. Die quadratische Laufzeit entsteht da pro Cache-Knoten die relativen Pfade errechnet werden müssen. Bei großen Welten mit vielen Details kann b_f wesentlich größer als 2 sein und Faktoren im Bereich von $10 - 100$ sind nicht unrealistisch.

5.2 Reduktion der Neuberechnungszeit

Um die Reduktion der Neuberechnungszeit zu bewerten wird untersucht, wie stark sich die Laufzeit im Vergleich zu einer vollständigen Neugenerierung reduziert.

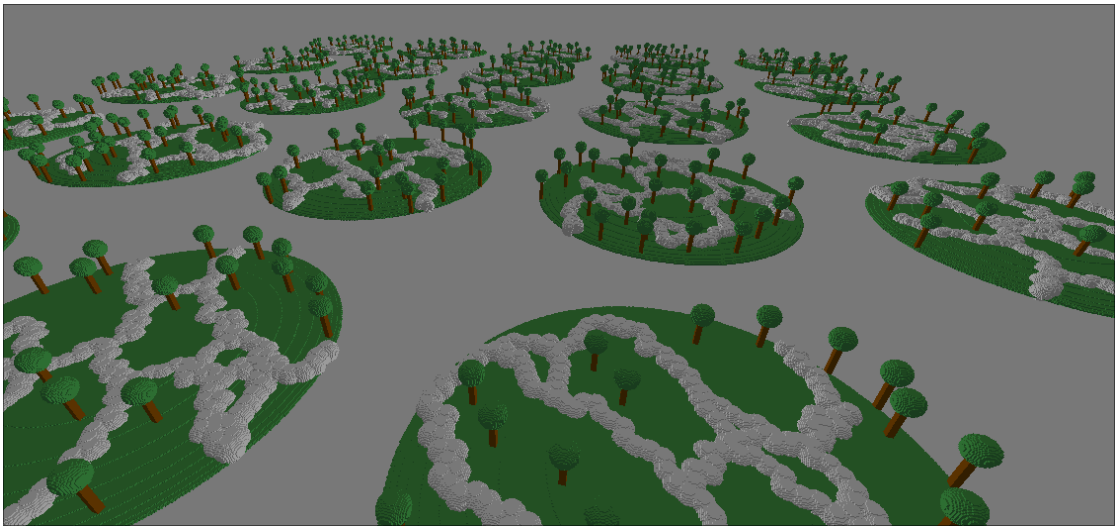

Abbildung 6:

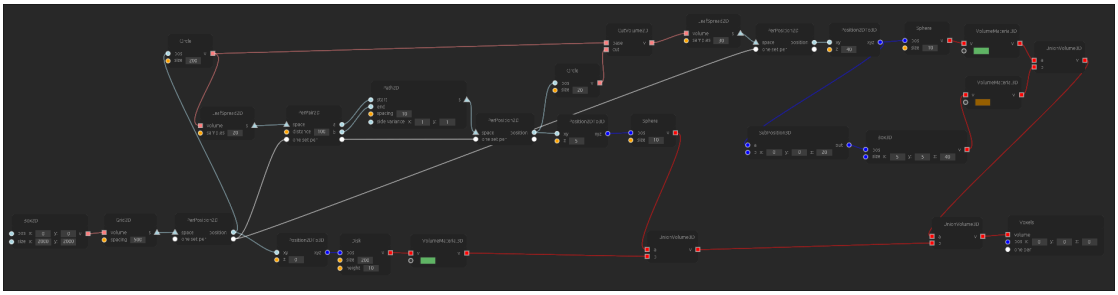

Abbildung 7:

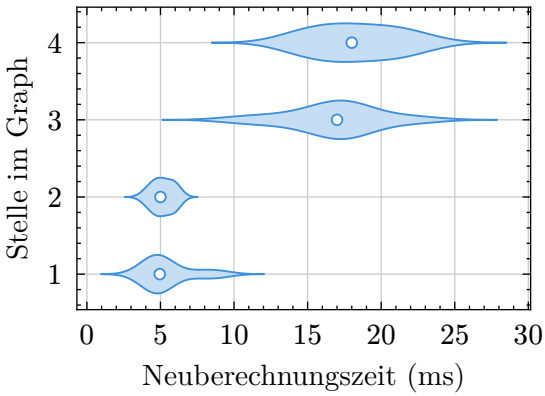

Abbildung 8: Unterschied der Neuberechnungszeit zwischen verschiedenen geänderten Stellen im Graph

Stellen:

- 4 ganz linke Box2D Node
- 3 mittlere Path2D Node
- 2 untere Disk Node
- 1 Sphere Node der Baum Krone auf der rechten Seite

In diesem Beispiel sieht man das Neuberechnungszeit nicht linear abfällt, sonder alle Änderungen wo die Wege und Bäume neu berechnet werden müssen im Durchschnitt 17ms benötigen wohin Änderungen die nur das finale Volumen betreffen um Durchschnitt 5ms benötigen. Somit lässt sich davon ausgehen das die Berechnung der Wege und Bäume ca. 12ms benötigt, welche weg fallen wenn die zwischengespeicherten Daten verwendet werden.

5.3 Overhead

Ein weiteres wichtiges Kriterium ist der Overhead des Systems. Da der Generationsalgorithmus nicht direkt als kompilierten Programmcode ausgeführt wird, sondern als Abhängigkeitsgraph interpretiert wird, entstehen zusätzliche Kosten. Diese entstehen insbesondere durch:

- die Verwaltung und Modifikation der Graphstruktur,
- das rekursive Auflösen der Abhängigkeiten während der Berechnung.

Hierfür wurde das Beispiel **TODO:** Name in zwei weiteren Versionen implementiert.

In der ersten würde die Generator-Graph-Verwaltungs-Logik entfernt. Hier wird der Abhängigkeits-Graph direkt evaluatiert ohne Zwischenspeicher anzulegen oder zu verwalten. Diese Änderunge reduziert zwar die Menge an Code signifikant jedoch hat keine großen Auswirkungen auf die Laufzeit. Dies hat wahrscheinlich Folgende Gründe: Die in Zwischenspeicher genutzten Mengen müssen bei der evaluation des Graphs eh angelegt werden und werden in diesem Fall nur wieder deallokiert anstatt weiterhin gespeichert zu bleiben.

Als zweites wurde eine Version des Generationsalgorithmus ohne Abhängigkeits-Graph direkt implementiert. Diese Version ist ca. $3-6$x schneller als das Beispiel. Dies hat wahrscheinlich mit dem Overhead durch die evaluation des Abhängigkeits-Graph zu tun. Einerseits können durch direkte Kapselung von loops die Allozierung und Erstellung von Mengen Vektoren gespart werden. Der Abhängigkeits-Graph benötigt durch seine Polymorphe-Natur viele Switch-Statements um zwischen den verschiedenen Funktionen die einen Wert erzeugen dynamisch zu unterschieden. Dies fällt bei einer direkten Implementierung weg. Dazu kann dadurch der Generationsalgorithmus könnte stärker durch den Compiler optimiert werden.

5.4 Nutzerfreundlichkeit

Durch den grafischen Editor kann der Generationsprozess als Abhängigkeits-Graph visualisiert und direkt bearbeitet werden. Dadurch lassen sich komplexe Zusammenhänge zwischen einzelnen Operationen häufig leichter nachvollziehen als in klassischem Quellcode, da Datenflüsse und Abhängigkeiten explizit dargestellt sind.

Insbesondere bei der experimentellen Entwicklung von Generationsalgorithmen kann dieser Ansatz Vorteile bieten. Änderungen am Graphen können direkt im Editor vorgenommen werden, ohne dass der gesamte Algorithmus neu implementiert werden muss. In Kombination mit der minimalen Neuberechnung des Generators können Anpassungen am Generationsprozess schnell getestet werden, da nur die betroffenen Teile der Welt neu berechnet werden.

Gleichzeitig hängt die wahrgenommene Nutzerfreundlichkeit stark vom jeweiligen Nutzertyp ab. Nutzer mit viel Erfahrung in klassischer Softwareentwicklung bevorzugen häufig eine direkte Implementierung in Quellcode, da diese mehr Kontrolle bietet und weniger strukturelle Einschränkungen hat. Für weniger erfahrene Nutzer oder für Anwender, die primär konzeptionell arbeiten möchten, kann ein grafischer Editor hingegen zugänglicher sein.

Die visuelle Darstellung der Abhängigkeiten erleichtert das Verständnis des Systems und erlaubt es, Generationslogik zu definieren, ohne sich stark mit den Details einer Programmiersprache beschäftigen zu müssen.

Allerdings kann ein solcher Editor auch einschränkend wirken, da nur die im System vorgesehenen Operationen und Strukturen genutzt werden können.

Insgesamt bietet der grafische Ansatz insbesondere für komplexe prozedurale Systeme Vorteile, da er eine bessere Übersicht über den Generationsprozess ermöglicht und iterative Änderungen am Algorithmus erleichtert, während klassische Code-basierte Ansätze weiterhin mehr Flexibilität für erfahrene Entwickler bieten.

5.5 Erweiterbarkeit

Um das System um weitere Operationen oder oder Datentype zu erweitern ist ein gutes Verständnis der Codestruktur benötigt. Jedoch sind keine grundlegenden Änderungen nötigt. Der Editor, das Template und der Generator stellen keine Erwartungen an die Art der Datentypen oder Operationen im Abhängigkeits Graph. Die Datentypen und Operationen sind mit Typed Unions implement daher können diese einfach erweitert werden. Jedoch muss die neue Variation an jeder Stelle wo ein Datentyp oder Operation allgemein verwendet wird implementiert werden um sie in das bestehende Netz an möglichen Abhängigkeiten zu intrigieren.

6 Future Work

6.1 Nullwerte führen zu leeren Lösungen

Da der Null Wert per definition eine valider Wert ist kann es dazu kommen, dass sich ein Abhängigkeits-Kreis zu null als Lösung entwickelt, auch wenn es theoretisch andere Lösungen gebe. Um dies zu lösen müsste ein anderer Ansatz zur Lösung von Abhängigkeits-Kreisen genutzt werden.

Arbeiten in Richtung closely connected Components in Verbindung könnten hier eine Lösung sein.

6.2 Andere Datenstrukturen

Wie schon in Abschnitt 4.6.4 beschrieben eignen sich CSGs nicht zum direkten rendering. Die Umwandlung zu einem Mesh mit „marching cubes“ ist aufwendig und benötigt wesentlich mehr Leistung benötigen, als die Generation selbst. Somit besteht weiterhin die Forschungsfrage in wie weit minimale Neuberechnung von prozeduralen Meshen direkt möglich ist.

6.3 Nutzerfreundlichkeitsstudie

Die Nutzerfreundlichkeit des Systems wurde in dieser Arbeit bisher nur qualitativ betrachtet. Eine fundierte Bewertung sollte durch strukturierte Nutzerstudien oder Experteninterviews ergänzt werden. Dabei wäre insbesondere interessant, wie Nutzer mit unterschiedlichem Erfahrungshintergrund mit dem grafischen Editor umgehen. Erfahrene Entwickler könnten den Editor als zu einschränkend empfinden, da nur die im System vorgesehenen Operationen genutzt werden können, während weniger erfahrene Nutzer von der visuellen Darstellung der Abhängigkeiten profitieren könnten. Ein konkreter Ansatz zur Verbesserung der Nutzerfreundlichkeit wäre die Einführung von aussagekräftigen Fehlermeldungen und Warnungen im Editor. Aktuell gibt das System wenig Rückmeldung darüber, warum ein bestimmter Generationsschritt fehlschlägt oder ein unerwartetes Ergebnis liefert. Eine bessere Visualisierung des Generationsprozesses, beispielsweise durch das Hervorheben von Knoten die neu berechnet werden, könnte dem Nutzer helfen nachzuvollziehen welche Teile der Welt von einer Änderung betroffen sind.

6.4 Parallelisierung der Generierung

In der aktuellen Implementation werden unabhängige Knoten im Abhängigkeitsgraph sequenziell abgearbeitet. Da Knoten desselben Levels keine gegenseitigen Abhängigkeiten haben, könnten diese grundsätzlich parallel berechnet werden. Eine systematische Parallelisierung der Levelweise Berechnung könnte die Generierungszeit bei komplexen Algorithmen mit vielen unabhängigen Operationen erheblich reduzieren. Dabei müsste jedoch sichergestellt werden, dass Schreibzugriffe auf den Generator-Graphen threadsicher sind, ohne dabei zu viel Overhead durch Synchronisation zu benötigen.

6.5 Automatische Cache-Optimierung

Aktuell muss der Nutzer manuell entscheiden, welche Knoten im Abhängigkeitsgraph gecacht werden sollen. Ein zu sparsames Caching führt dazu, dass bei Änderungen viele Knoten neu berechnet werden müssen, während ein zu großzügiges Caching unnötig viel Speicher verbraucht und den Overhead erhöht.

Ein interessanter Forschungsansatz wäre ein System, das diese Entscheidung automatisch trifft. Dazu könnte der Generator zur Laufzeit messen, wie häufig ein Knoten wiederverwendet wird und wie aufwendig seine Berechnung ist. Basierend auf diesen Metriken könnte das System dynamisch entscheiden, welche Knoten gecacht werden sollen.

6.6 Skalierung auf sehr große Welten

Die vorliegende Arbeit evaluiert das System anhand vergleichsweise kleiner Beispielwelten. Es bleibt offen, wie das System mit sehr großen Welten skaliert, die aus tausenden gleichzeitig aktiver Chunks und sehr tiefen Abhängigkeitsgraphen bestehen. Bei sehr großen Welten könnte dies zu erheblichem Speicherverbrauch führen. Zukünftige Arbeiten sollten untersuchen, wie ein solches System auf gesamten Spielwelten agiert.

6.7 Integration in bestehende Engines

Diese Arbeit stellt ein System vor welches Spiel- oder Simulations-Engine einbettbar sein soll. Jedoch wurde die Frage einer Integration in eine bestehende Game Engine nicht behandelt.

Zukünftige Arbeiten sollten untersuchen, welche Schnittstellen eine Engine bereitstellen muss und welcher Integrationsaufwand in bestehende Engines wie Unity oder Unreal Engine entsteht. Dabei sind insbesondere die Synchronisation zwischen dem Generator und dem Render-Thread sowie die Übergabe extern kontrollierter Variablen wie Kameraposition oder Spielzustand relevante Fragestellungen.

6.8 Serialisierung und Persistenz

In der aktuellen Implementation existieren gecachte Zwischenergebnisse nur im Arbeitsspeicher und gehen beim Beenden der Anwendung verloren. Für einen praktischen Einsatz wäre es jedoch hilfreich, den Zustand des Generator-Graphen auf Disk zu speichern. So könnten bereits berechnete Teile der Welt beim nächsten Start der Anwendung wiederverwendet werden, ohne sie vollständig neu berechnen zu müssen. Dabei stellen sich Fragen zur effizienten Serialisierung großer Graphstrukturen sowie zur Invalidierung gespeicherter Ergebnisse, wenn sich das Template zwischen zwei Sitzungen geändert hat.

7 Fazit

In dieser Arbeit wurde gezeigt, dass minimale Neuberechnung für prozedurale Generierung grundsätzlich möglich ist. Die Beispielimplementation zeigt, dass sich ein Generationsalgorithmus als Abhängigkeitsgraph darstellen lässt und Zwischenergebnisse gezielt wiederverwendet werden können. Ändert sich ein Teil des Algorithmus, müssen nur die betroffenen Knoten neu berechnet werden.

Jedoch da der Algorithmus als Graph interpretiert wird statt direkt als kompilierten Code ausgeführt zu werden, ist er deutlich langsamer als eine direkte Implementierung. In den Benchmarks war eine optimierte direkte Implementierung etwa 3 - 6x schneller. Dieser Overhead entsteht vor allem durch das rekursive Auflösen der Abhängigkeiten und die polymorphe Natur des Graphen. Dazu kommt, dass der Aufwand zur Implementierung aller benötigten Operationen und Datentypen.

Trotzdem bietet mein Ansatz mein ein graphisches Interface um Prozedurale Generationsalgorithen interaktiv zu entwickeln. Es bietet eine vereinfachte Abstraktions-Ebene zum testen von Generationslogik die keine Wissen über Programmiersprachen benötigt.

Für einen realen Einsatz in einem Spiel oder einer Simulation würde ist mein Ansatz sinnvoll, wenn der Generationsalgorithmus für eine prozedurale Welt Personen von entwickelt werden soll die nicht umfassend programmieren können und sehr interaktiv jede Änderung testen wollen.

8 Bibliografie

[1] P. Merrell, „Example-based model synthesis“, in *Proceedings of the 2007 Symposium on Interactive 3D Graphics and Games*, in I3D '07. Seattle, Washington: Association for Computing Machinery, 2007, S. 105–112. doi: 10.1145/1230100.1230119.

[2] V. Volz, J. Schrum, J. Liu, S. M. Lucas, A. Smith, und S. Risi, „Evolving Mario Levels in the Latent Space of a Deep Convolutional Generative Adversarial Network“. [Online]. Verfügbar unter: https://arxiv.org/abs/1805.00728

[3] M. Bazzaz und S. Cooper, „Level Generation with Constrained Expressive Range“, in *Proceedings of the 20th International Conference on the Foundations of Digital Games*, in FDG '25.: Association for Computing Machinery, 2025. doi: 10.1145/3723498.3723845.

[4] O. Withington und L. Tokarchuk, „Compressing and Comparing the Generative Spaces of Procedural Content Generators“, in *2022 IEEE Conference on Games (CoG)*, 2022, S. 143–150. doi: 10.1109/CoG51982.2022.9893615.

[5] A. Goslin, „Terrain Diffusion: A Diffusion-Based Successor to Perlin Noise in Infinite, Real-Time Terrain Generation“. [Online]. Verfügbar unter: https://arxiv.org/abs/2512.08309

[6] B. Bollobás, *Modern Graph Theory*. Springer New York, 2013. doi: 10.1007/978-1-4612-0619-4.

[7] A. A. G. Requicha und H. B. Voelcker, „Constructive Solid Geometry“, Rochester, NY, USA, technical report, 1977.

[8] B. Wassermann, S. Kollmannsberger, T. Bog, und E. Rank, „From geometric design to numerical analysis: A direct approach using the Finite Cell Method on Constructive Solid Geometry“, *Computers & Mathematics with Applications*, Bd. 74, Nr. 7, S. 1703–1726, Okt. 2017, doi: 10.1016/j.camwa.2017.01.027.

[9] O. Peters, „slotmap: A Rust library for stable keys and values“. [Online]. Verfügbar unter: https://crates.io/crates/slotmap

[10] C. A. R. Hoare, „Communicating Sequential Processes“, *Communications of the ACM*, Bd. 21, Nr. 8, S. 666–677, 1978, doi: 10.1145/359576.359585.

[11] smol-rs developers, „async-channel: Async multi-producer multi-consumer channel“. [Online]. Verfügbar unter: https://docs.rs/async-channel

[12] S. P. Developers, „smallvec: A vector that can store a small number of elements inline“. [Online]. Verfügbar unter: https://crates.io/crates/smallvec

[13] J. Amanatides und A. Woo, „A Fast Voxel Traversal Algorithm for Ray Tracing“, *Proceedings of EuroGraphics*, Bd. 87, S. , 1987.

[14] S. Laine und T. Karras, „Efficient Sparse Voxel Octrees“, *IEEE Transactions on Visualization and Computer Graphics*, Bd. 17, Nr. 8, S. 1048–1059, 2011, doi: 10.1109/TVCG.2010.240.

[15] W. E. Lorensen und H. E. Cline, „Marching cubes: A high resolution 3D surface construction algorithm“, *SIGGRAPH Comput. Graph.*, Bd. 21, Nr. 4, S. 163–169, Aug. 1987, doi: 10.1145/37402.37422.

Universität Bremen

Eigenständigkeits- und Einverständniserklärung zur Überprüfung mit Plagiatssoftware sowie die Erklärung zur Veröffentlichung bei Bachelor- und Masterarbeiten

Declarations of Authorship and Consent for Checking with Plagiarism Software and the Declaration of Publication for Bachelor's and Master's Thesis

Studierenden-Angaben / Student Information:

Matrikelnr./ Student ID	Nummer
Nachname / Surname	Behn
Vorname / First Name	Maarten

Titel der Arbeit / Title of Thesis

Minimale Neuberechnung Abhängigkeits-Graph basierter Regeln zur prozeduralen Welten-Generation.

A) Eigenständigkeitserklärung / Declaration of Authorship

Ich versichere, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Teile meiner Arbeit, die wortwörtlich oder dem Sinn nach anderen Werken entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht. Gleiches gilt auch für Zeichnungen, Skizzen, bildliche Darstellungen sowie für Quellen aus dem Internet, dazu zählen auch KI-basierte Anwendungen oder Werkzeuge.
Die Arbeit wurde in gleicher oder ähnlicher Form noch nicht als Prüfungsleistung eingereicht.

I hereby affirm that I have written the present work independently and have used no sources or aids other than those indicated. All parts of my work that have been taken from other works, either verbatim or in terms of meaning, have been marked as such, indicating the source. The same applies to drawings, sketches, pictorial representations and sources from the Internet, including AI-based applications or tools. The work has not yet been submitted in the same or a similar form as a final examination paper.

☒ Ich habe KI-basierte Anwendungen und/oder Werkzeuge genutzt und diese im Anhang "Nutzung KI basierte Anwendungen" dokumentiert.

I have used AI-based applications and/or tools and documented them in the appendix "Use of AI-based applications".

B) **Erklärung zur Veröffentlichung von Bachelor- oder Masterarbeiten**
Declaration regarding the publication of bachelor's or master's thesis

Die Abschlussarbeit wird zwei Jahre nach Studienabschluss dem Archiv der Universität Bremen zur dauerhaften Archivierung angeboten. Archiviert werden:

Two years after graduation, the thesis is offered to the archive of the University of Bremen for permanent archiving. The following are archived:

1) Masterarbeiten mit lokalem oder regionalem Bezug sowie pro Studienfach und Studienjahr 10 % aller Masterarbeiten
Master's theses with a local or regional focus, as well as per subject and academic year 10% of all Master's thesis

2) Bachelorarbeiten des jeweils ersten und letzten Bachelorabschlusses pro Studienfach und Jahr.
Bachelor's thesis for the first and last bachelor's degrees per subject and year.

☒ Ich bin damit einverstanden, dass meine Abschlussarbeit im Universitätsarchiv für wissenschaftliche Zwecke von Dritten eingesehen werden darf.
I agree that my thesis may be viewed by third parties in the university archive for academic purposes.

☒ Ich bin damit einverstanden, dass meine Abschlussarbeit nach 30 Jahren (gem. §7 Abs. 2 BremArchivG) im Universitätsarchiv für wissenschaftliche Zwecke von Dritten eingesehen werden darf.
I agree that my thesis may be viewed by third parties for academic purposes in the university archive after 30 years (in accordance with §7 para. 2 BremArchivG).

☐ Ich bin **nicht** damit einverstanden, dass meine Abschlussarbeit im Universitätsarchiv für wissenschaftliche Zwecke von Dritten eingesehen werden darf.
I do not consent to my thesis being made available in the university archive for third parties to view for academic purposes.

C) Einverständniserklärung zur elektronischen Überprüfung der Arbeit auf Plagiate
Declaration of consent for electronic checking of the work for plagiarism

Eingereichte Arbeiten können nach § 18 des Allgemeinen Teil der Bachelor- bzw. der Masterprüfungsordnungen der Universität Bremen mit qualifizierter Software auf Plagiatsvorwürfe untersucht werden.
Zum Zweck der Überprüfung auf Plagiate erfolgt das Hochladen auf den Server der von der Universität Bremen aktuell genutzten Plagiatssoftware.

Submitted papers can be checked for plagiarism using qualified software in accordance with § 18 of the General Section of the Bachelor's or Master's Degree Examination Regulations of the University of Bremen. For the purpose of checking for plagiarism, the upload to the server is done using the plagiarism software currently used by the University of Bremen.

☒ Ich bin damit einverstanden, dass die von mir vorgelegte und verfasste Arbeit zum oben genannten Zweck dauerhaft auf dem externen Server der aktuell von der Universität Bremen genutzten Plagiatssoftware, in einer institutionseigenen Bibliothek (Zugriff nur durch die Universität Bremen), gespeichert wird.

I agree that the work I have submitted and written will be stored permanently on the external server of the plagiarism software currently used by the University of Bremen, in a library belonging to the institution (accessed only by the University of Bremen), for the above-mentioned purpose.

☐ Ich bin **nicht** damit einverstanden, dass die von mir vorgelegte und verfasste Arbeit zum o.g. Zweck dauerhaft auf dem externen Server der aktuell von der Universität Bremen genutzten Plagiatssoftware, in einer institutionseigenen Bibliothek (Zugriff nur durch die Universität Bremen), gespeichert wird.

I do not consent to the work I submitted and wrote being permanently stored on the external server of the plagiarism software currently used by the University of Bremen, in a library belonging to the institution (accessed only by the University of Bremen), for the above-mentioned purpose.

Das Einverständnis der dauerhaften Speicherung des Textes ist freiwillig. Die Einwilligung kann jederzeit durch Erklärung gegenüber der Universität Bremen, mit Wirkung für die Zukunft, widerrufen werden. Weitere Informationen zur Überprüfung von schriftlichen Arbeiten durch die Plagiatsoftware sind im Nutzungs- und Datenschutzkonzept enthalten. Diese finden Sie auf der Internetseite der Universität Bremen.

Consent to the permanent storage of the text is voluntary. Consent can be withdrawn at any time by making a declaration to this effect to the University of Bremen, with effect for the future. Further information on the checking of written work using plagiarism software can be found in the data protection and usage concept. This can be found on the University of Bremen website.

Mit meiner Unterschrift versichere ich, dass ich die obenstehenden Erklärungen gelesen und verstanden habe und bestätige die Richtigkeit der gemachten Angaben.

With my signature, I confirm that I have read and understood the above explanations and confirm the accuracy of the information provided.

Datum / Date Unterschrit/ Signature

9.1 Nutzung KI basierte Anwendungen

Neuronale Large Language Modells wurden zur Erstellung dieser Arbeit in Folgenden Bereichen verwendet:

- In der Recherche: Um einen Überblick über den Stand der Technik zu bekommen und Arbeiten zu finden die ich noch nicht kannte habe ich mir Empfehlungen für bestehende Arbeiten generien lassen.
- Rechtschreib- und Satzbaukorrektur: Eine von mir verfasste roh Fassung wurde von KI auf Rechtschreibung- und Satzbaukorrigiert. Dabei habe ich genau darauf geachtet sich der Inhalt vorgeschlagener Anpassung nicht geändert hat.

Bereiche in den keine KI verwendet wurden ist:

- Programmierung: Es wurde keine KI verwendet um dem Code meiner Implementation zu schreiben.
- Text Inhalte: KI wurde nicht verwendet um den Inhalt der Arbeit zu bestimmen.